

認識 Git 版本控管

Git 歷史

Git 的出現，來自於 Linux 之父 "Linus Torvalds" 開發 Linux kernel 的時候，因為早期的版本控制方法非常沒有效率，屬集中式控管，當 Linux kernel 這類複雜又龐大的專案在進行版本控管時，出現了許多問題。最早期 Linux kernel 採用 BitKeeper 進行版本控管，但後來 Linus Torvalds 基於 BitKeeper 與 Monotone 的使用經驗，設計出更棒的 Git 版控系統。原先 Git 只被設計成一個低階的版控工具，用來當做其他版控系統 (SCM) 的操作工具，後來才漸漸演變成一套完整的版本控制系統。

有趣的是，Linus Torvalds 改採 Git 進行版本控管初期，由於 Git 太過複雜，許多版控觀念跟以往差異太大，也受到世界各地開放原始碼社群的反對，但經過幾年的努力與發展，操作 Git 的相關工具也越來越成熟，才漸漸平撫反對的壓力，從 2013 年的市場調查看來，全世界已有 30% 的開放原始碼專案改採 Git 進行版本控管，這是個非常驚人的市占率，意謂著 Git 絕對有其驚豔之處，好好研究一番還不行呢！

講到 Git 的架構，完全是基於 Linus Torvalds 在維護 Linux kernel 這個大型專案時得到的經驗，以及他本身在檔案系統優化方面的豐富經驗進行設計，也因為這樣，Git 包含了以下幾個重要的設計：

- 強力支援非線性開發模式 (分散式開發模式)
 - Git 擁有快速的分支與合併機制，還包括圖形化的工具顯示版本變更的歷史路徑。
 - Git 非常強調分支與合併，所以版本控管的過程中，你會不斷的在執行分支與合併動作。
 - Git 的分支機制非常輕量，沒有負擔，每一次的分支只是某個 commit 的參考指標而已。
- 分散式開發模型
 - 參與 Git 開發的每個人，都將擁有完整的開發歷史紀錄。
 - 當開發人員第一次將 Git 版本庫複製 (clone) 下來後，完全等同於這份 Git 版本庫的「完整備份」。
 - 整個版本庫中所有變更過的檔案與歷史紀錄，通通都會儲存在本機儲存庫 (local repository)。
- 相容於現有作業系統
 - Git 版本庫其實就只是一個資料夾而已，資料夾中有許多相關的設定檔與各種 blob 物件檔案而已。
 - Git 版本庫可以用任何方式發布，所以你用 HTTP, FTP, rsync, SSH 甚至於用 Git protocol 都可以當成存取 Git 版本庫的媒介，相容性極高。

- 有效率的處理大型專案
 - 由於完整的版本庫會複製 (clone) 一份在本機，該版本庫包含完整的檔案與版本變更紀錄，所以針對版本控管中的各種檔案操作速度，將會比直接從遠端存取來的快上百倍之多。
 - 這也代表著，Git 版本控管不會因為專案越來越大、檔案越來越多，而導致速度變慢。
- 歷史紀錄保護
 - Git 版控的過程，每次 commit 都會產生一組 hash id 編號，而且每個版本在變化的過程都會參考到這個 hash id，只要 hash id 無法比對的上，Git 就會無法運作，所以當專案越來越大，版本庫複製 (clone) 的越來越多份，你幾乎無法竄改檔案的內容或版本紀錄。
 - 請記得：每個人都有一份完整的版本庫，你改了原始的那份，所有人的版本庫就無法再合併回原本的版本庫了，所以你幾乎不可能任意竄改版本紀錄。

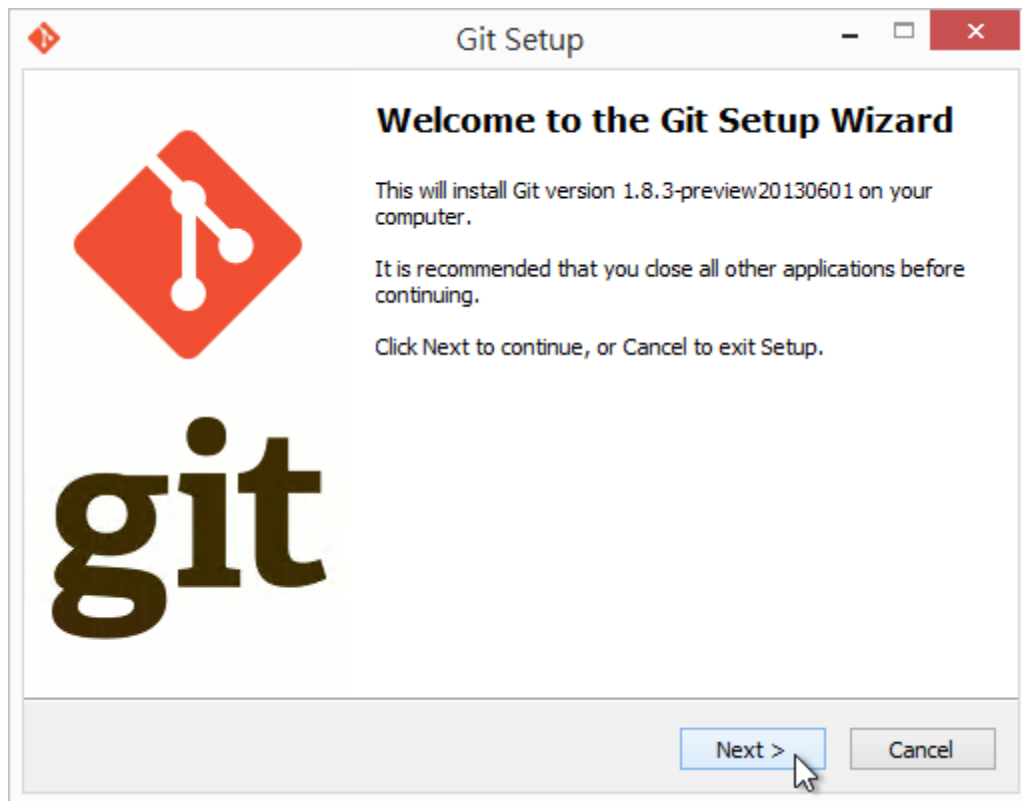
Install GIT

這是一個可以在 **命令提示字元** (Command Prompt) 下執行的一套指令列工具，包含 Git 簡易的 GUI 工具

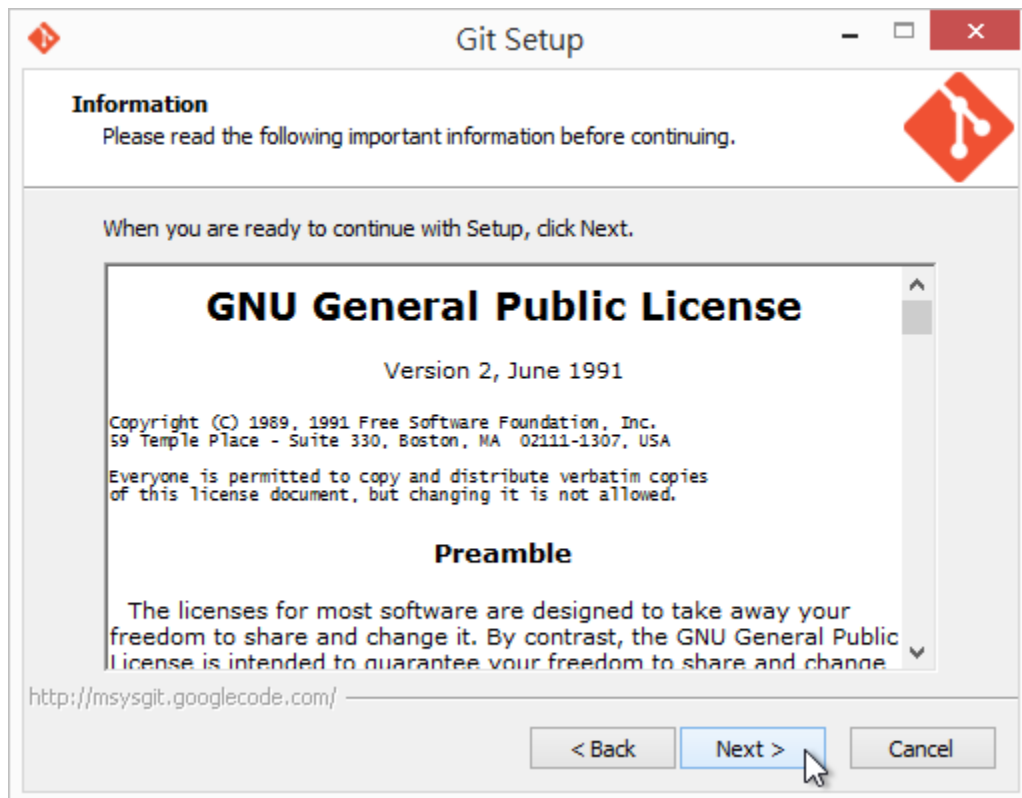
Download:

<https://git-scm.com/downloads>

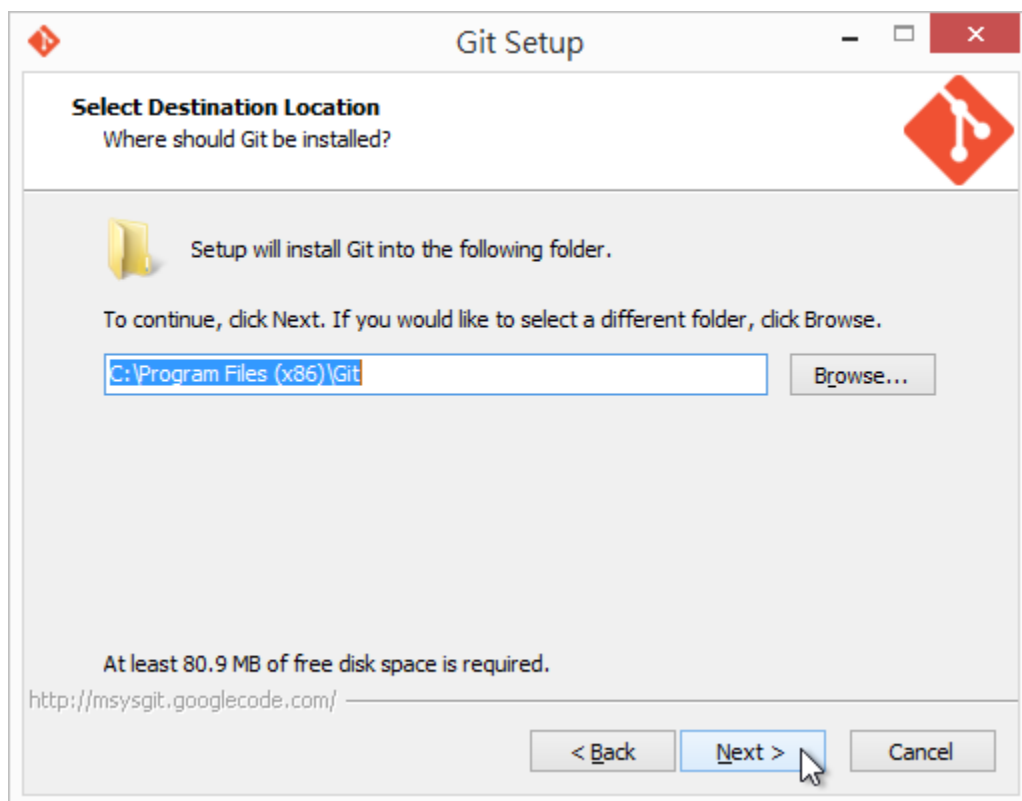
- 安裝程式歡迎頁面



- 同意 GPL 授權條款

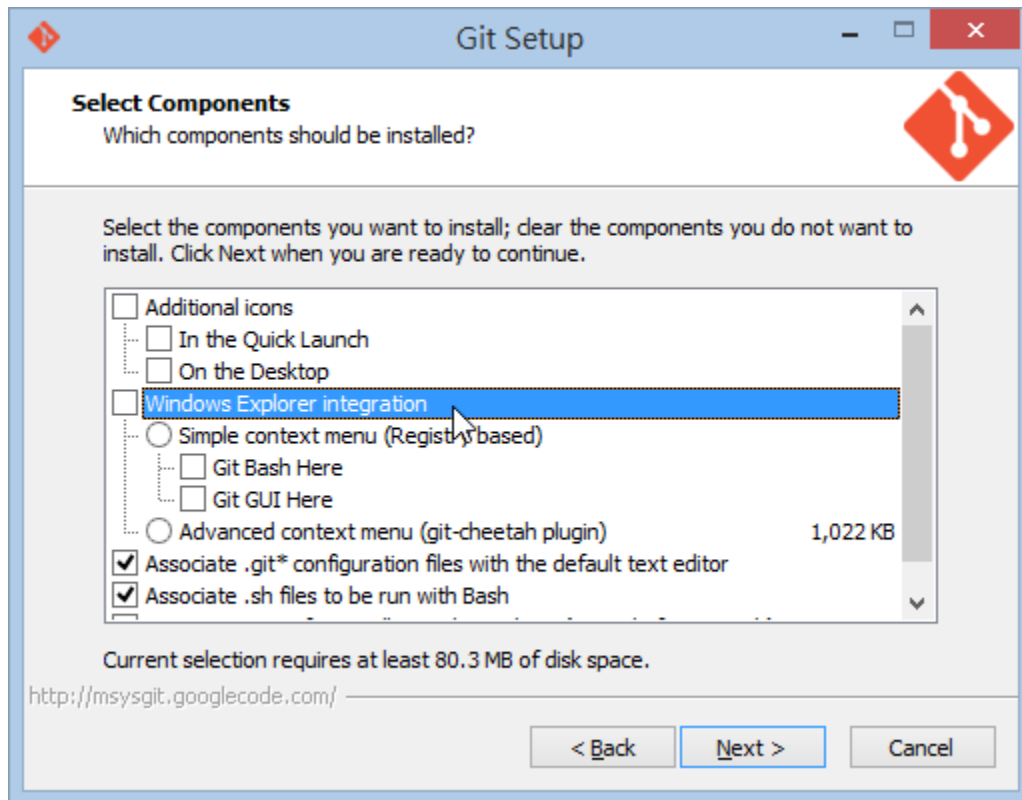


- 選擇安裝路徑

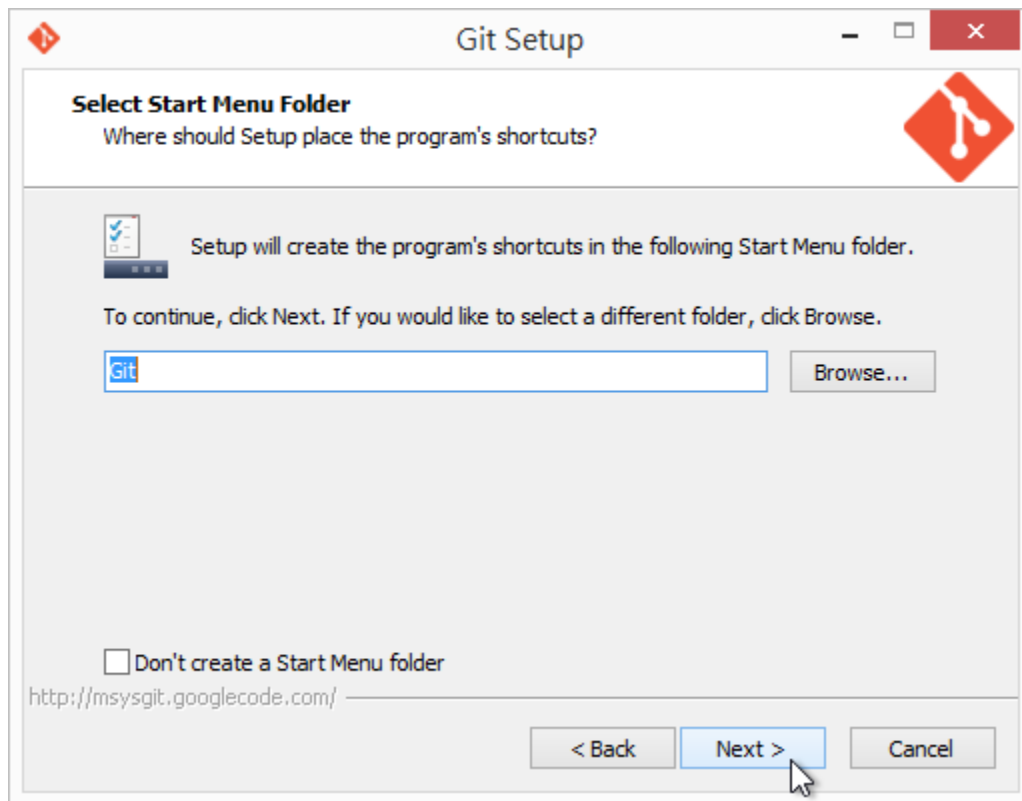


- 選取元件，建議取消勾選 Windows Explorer integration 選項！

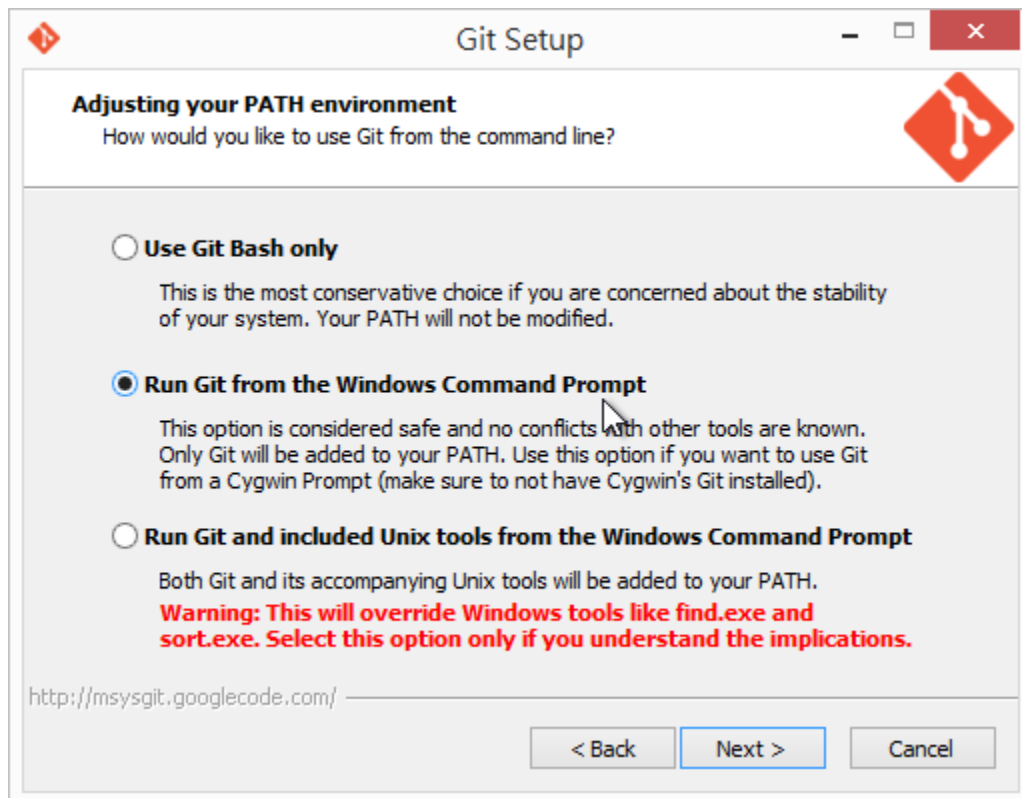
(因為這套整合工具太陽春，建議改用 [TortoiseGit](#) 工具)



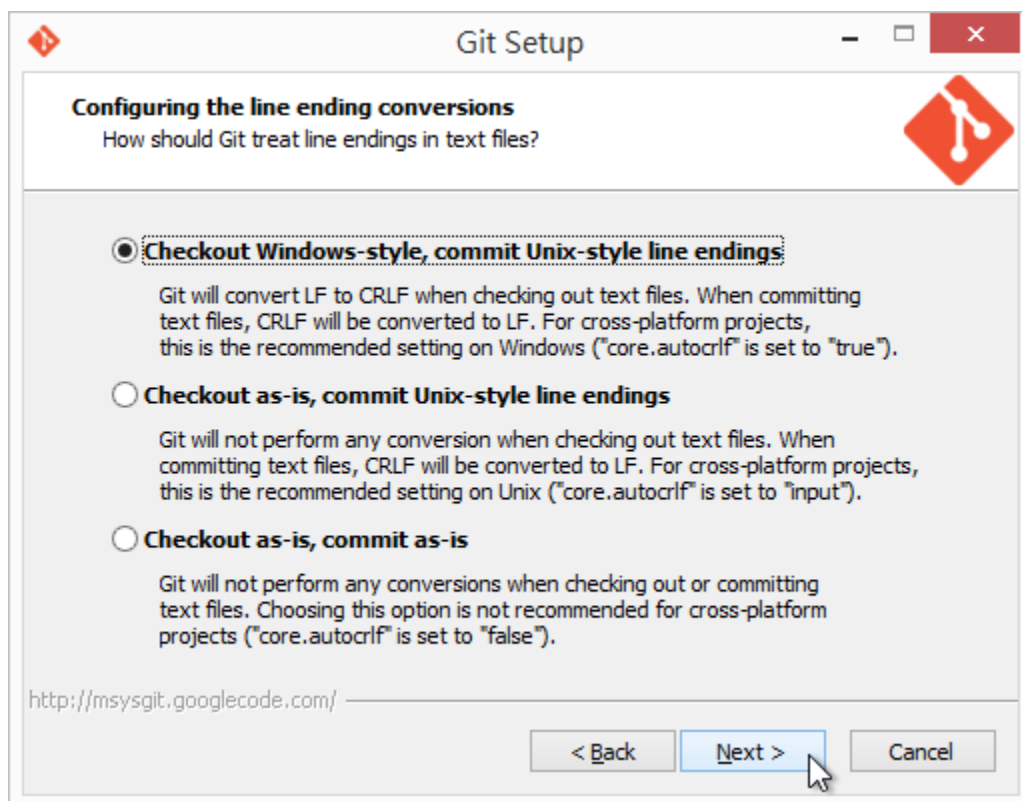
- 設定程式集名稱，保留預設選項即可



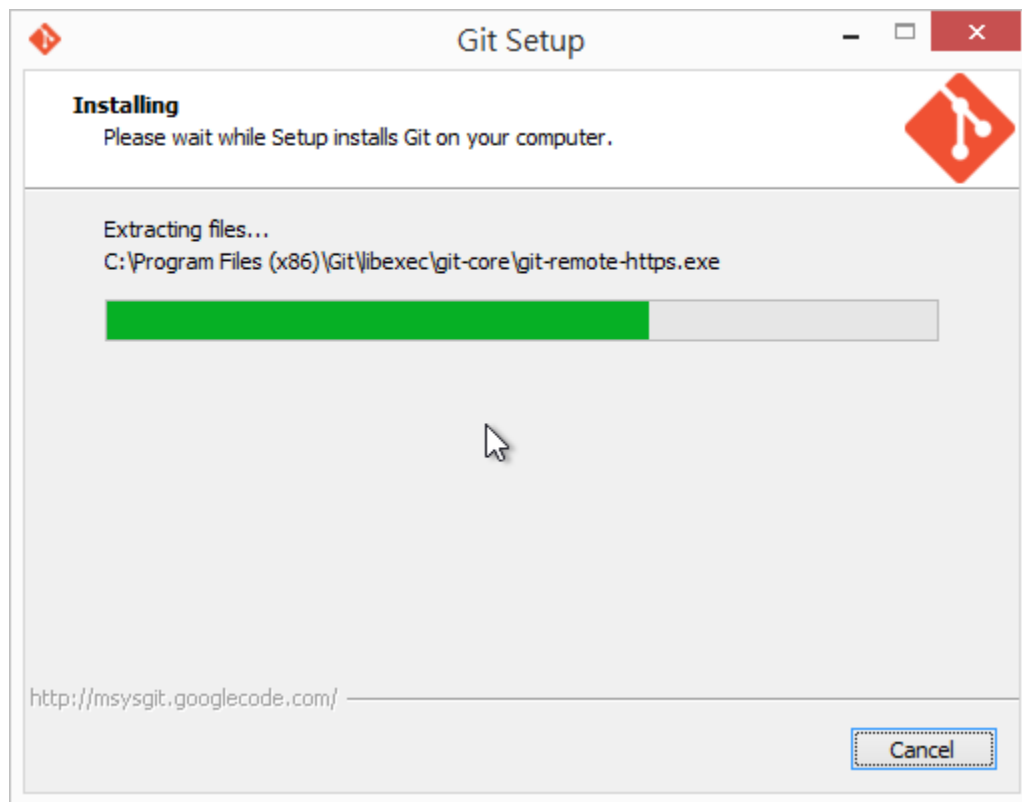
- 這裡建議 選擇選擇 Run Git from the Windows Command Prompt
這個選項！
(因為 Windows 使用者大多不熟 Bash 指令)



- 換行字元轉換，建議保留預設選項即可。



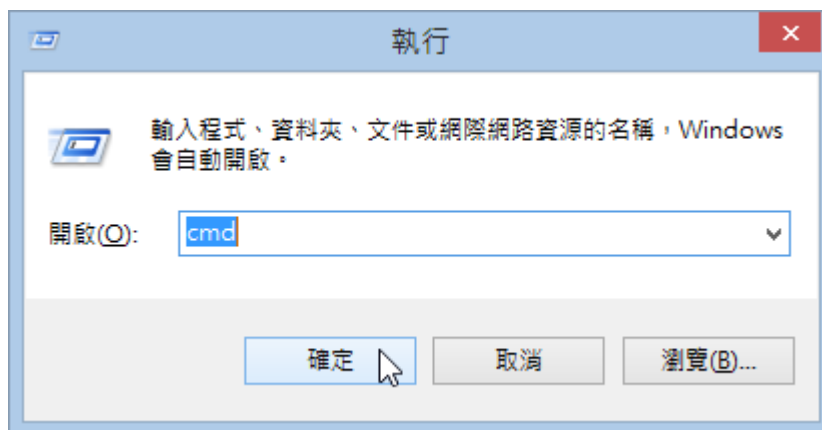
- 開始進行安裝



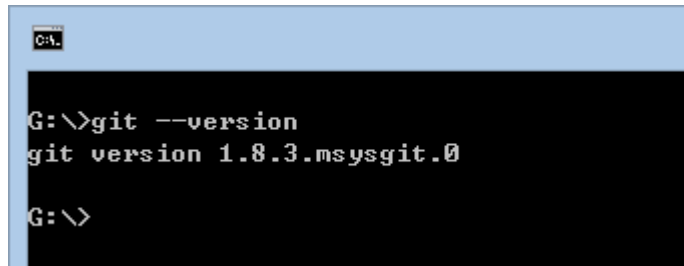
- 安裝完成



- 安裝完成後，直接開啟命令提示字元，就可以開始使用



- 你可以輸入 `git --version` 指令查詢目前工具程式的版本

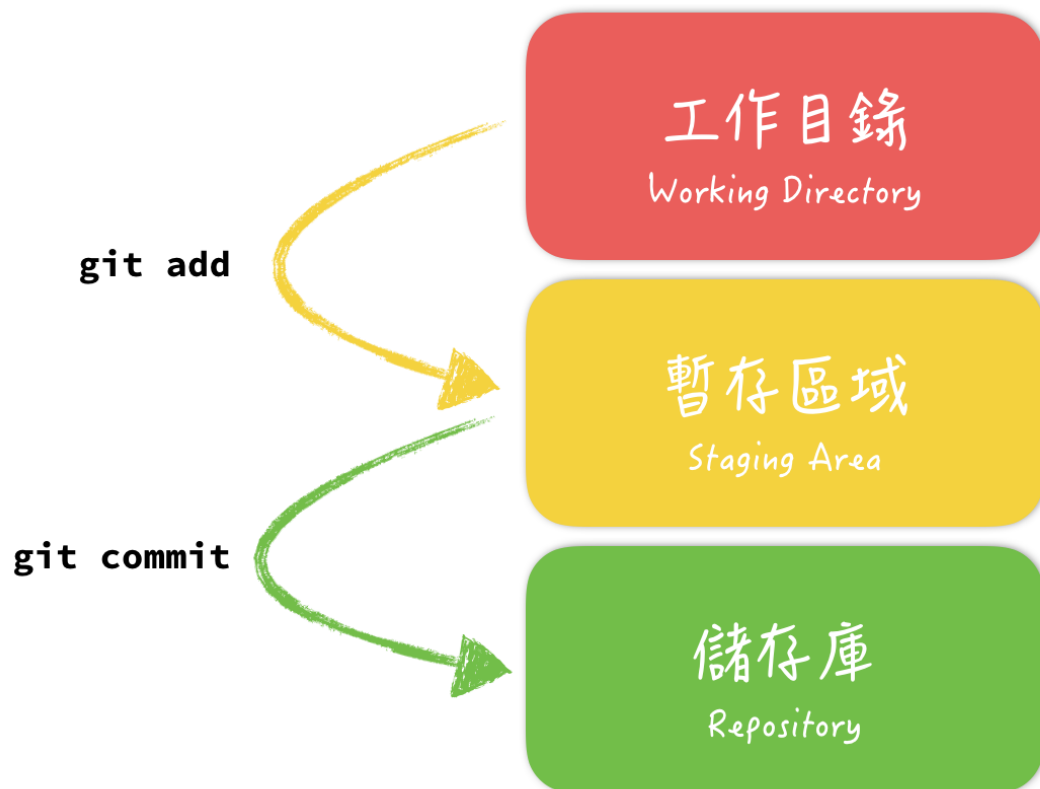
A screenshot of a Windows command prompt window. The title bar is light blue and contains the text 'C:\>'. The command prompt itself is black with white text. The first line shows the command 'G:\>git --version'. The second line shows the output 'git version 1.8.3.msysgit.0'. The third line shows the prompt 'G:\>' again.

```
G:\>git --version
git version 1.8.3.msysgit.0
G:\>
```

工作區、暫存區與儲存庫

可以使用 `git add` 指令把檔案加至暫存區 (Staging Area)，然後可使用 `git commit` 指令把暫存區的內容移往儲存庫 (Repository)。

在 Git 裡，主要可以分成「工作目錄(Working Directory)」、「暫存區(Staging Area)」以及「儲存庫 (Repository)」三個區塊，透過不同的 Git 指令，可以把檔案移往不同的區域：



1. `git add` 指令把檔案從工作目錄移至暫存區 (或索引)。
2. `git commit` 指令把暫存區的內容移至儲存庫。

了解儲存庫

我們要使用 Git 進行版本控管，很自然的，我們需要一個「版本庫」來儲存這些版本資訊，而英文的 `Repository` 就是這個意思，筆者習慣將這個英文翻譯成「儲存庫」，代表用來儲存所有版本的一個空間或一個資料夾與一堆檔案。

如果有 Git 使用經驗的人，應該很清楚，建立儲存庫有很多方法，如果你要在任意一個資料夾建立一個 Git 儲存庫，只要輸入以下指令就可以建立完成：

`git init`

我們透過下圖建立 Git 儲存庫的過程來說明，透過這張圖我們可以很清楚的知道，當我們在 `G:\git-demo` 目錄下執行 `git init` 之後，Git 會自動幫我們建立一個所謂的 Git repository 在該目錄的 `.git` 目錄下，各位不用懷疑，這個 `.git` 資料夾，就是一個完整的 Git 儲存庫，未來所有版本的變更，都會自動儲存在這個資料夾裡面。

```
C:\Windows\
G:\>mkdir git-demo

G:\>cd git-demo

G:\git-demo>git init
Initialized empty Git repository in G:/git-demo/.git/

G:\git-demo>dir /a
磁碟區 G 中的磁碟是 TEMPORARY
磁碟區序號: 02EB-0000

G:\git-demo 的目錄

2013/09/14 下午 03:51 <DIR> .
2013/09/14 下午 03:51 <DIR> ..
2013/09/14 下午 03:51 <DIR> .git
0 個檔案 0 位元組
3 個目錄 4,195,934,208 位元組可用

G:\git-demo>dir .git\
磁碟區 G 中的磁碟是 TEMPORARY
磁碟區序號: 02EB-0000

G:\git-demo\.git 的目錄

2013/09/14 下午 03:51 <DIR> .
2013/09/14 下午 03:51 <DIR> ..
2013/09/14 下午 03:51 <DIR> refs
73 description
2013/09/14 下午 03:51 <DIR> hooks
2013/09/14 下午 03:51 <DIR> info
2013/09/14 下午 03:51 23 HEAD
2013/09/14 下午 03:51 157 config
2013/09/14 下午 03:51 <DIR> objects
3 個檔案 253 位元組
6 個目錄 4,195,934,208 位元組可用

G:\git-demo>
```

了解工作目錄

在上述這個例子裡，目錄 `G:\git-demo` 此時就會自動成為我們的「工作目錄」(working directory)。所謂「工作目錄」的意思，就是我們正在準備開發的專案檔案，未來都會在這個目錄下進行編輯，無論是新增檔案、修改檔案、刪除檔案、

檔案更名、...以及所有其他 Git 相關的操作，都會在這個目錄下完成，所以才稱為「工作目錄」。

我們在操作 Git 相關指令參數的時候，也通常都是在「工作目錄」下執行的。

由於在使用 Git 版本控管時，會遭遇到很多分支的狀況，所以工作目錄很有可能會在不同的分支之間進行切換，有些 git 指令在執行的時候，會一併更新工作目錄下的檔案。例如當你使用 `git checkout` 切換到不同分支時，由於目前分支與想要切換過去的分支的目錄結構不太一樣，所以很有可能會將你目前工作目錄下的檔案進行更新，好讓目前的工作目錄下的這些目錄與檔案，都與另一個要切換過去的分支下的目錄與檔案一樣。

Git 指令表

config

Git	zsh	do	Remark
<code>git config --list</code>		查看設定	
<code>git config --local user.name "(userName)"</code>		設定帳號	

Git	zsh	do	Remark
<code>git config --local user.email "(e-mail)"</code>		設定 E-mail	全域
<code>git config --global user.name "(userName)"</code>		設定帳號	單專案
<code>git config --global user.email "(e-mail)"</code>		設定 E-mail	單專案

init / clone

Git	zsh	do	Remark
<code>git clone</code>		抓遠端儲存庫下來	
<code>git init</code>		Git 初始化	
<code>rm -rf .git</code>		移除 Git	

remote

Git	zsh	do	Remark
<code>git remote add (origin) (git@~.git)</code>		遠端連結	
<code>git remote set-url (origin) (git@~.git)</code>	-	修改遠端連結	
<code>git remote remove (origin)</code>	-	移除遠端連結	
<code>git remote -v</code>		查詢遠端連結(URL)	
<code>git push -u (origin) (master)</code>		推上遠端並綁定	

基本版更 (pull / push / add / commit / status)

Git	zsh	do	Remark
<code>git status</code>	gst		

Git	zsh	do	Remark
git add (file)	ga (~)		
git add .	ga .		
git commit -m'message'	gcmmsg '(~)'		
git pull	gl		
git push (remote) (branch)	gp (~) (~)		
git push -u (remote) (branch)			
git restore --staged (file)		取消 git add	
git pull --rebase (remote) (branch)	gl		

檔案變更版更操作

Git	zsh	do	Remark
<code>git clean -fd</code>	-	清除未被追蹤的所有檔案	已編輯的會恢復,新增的不會變動
<code>git checkout (file)</code>	-	當前目錄回復前次存檔	(已編輯的會恢復,新增的不會變動)
<code>git restore (file)</code>	-	當前目錄回復前次存檔	(含被刪除的檔)

Branch 分支應用

Git	zsh	do	Remark
<code>git branch</code>	-	查詢所有本地分支	
<code>git branch -a</code>	-	查詢所有遠端分支	

Git	zsh	do	Remark
<code>git branch (newBranch)</code>	-	當前 commit 新建分支	
<code>git branch (newBranch) (commitID)</code>	-	特定 commit 上新建分支	
<code>git checkout (branch)</code>	-	切換到某分支	
<code>git checkout -b (newBranch)</code>	-	新建分支並切換過去	
<code>git branch -d (branch)</code>	-	刪除某分支	
<code>git branch -D (branch)</code>	-	強制刪除某分支	
<code>git branch -m (branch) (newName)</code>	-	將某 branch 更名	必須先切到不同分支

Reset 切到某版本

Git	zsh	do	Remark
<code>git reset (commit)</code>			預設為'mixed'
<code>git reset (commit) --mixed</code>			放回"1-工作目錄"
<code>git reset (commit) --soft</code>			放回"2-暫存區"
<code>git reset (commit) --hard</code>			都不留(直接被隱藏)
<code>git reset (commit)^</code>		退回前 1 次的 commit	^^ 退回前 2 版...
<code>git reset (commit)~5</code>		退回前 5 次的 commit	~N 退至前 N 版

- (commit)可以是 branch / commit ID / HEAD

Rebase & Merge 合併應用

Git	zsh	do	Remark

Git	zsh	do	Remark
git rebase (branch)		重接分支基底	
git merge (branch)		合併分支(平行)	

查詢

Git	zsh	do	Remark
git config --list		查詢目前設定	
which git		查詢 Git 位置	
git --version		查詢 Git 版本	
git status		查詢狀態	
git log		查詢 Log	

Git	zsh	do	Remark
<code>git log --oneline</code>		查詢 Log(單行顯示)	
<code>git log --oneline --all</code> <code>--graph</code>		樹狀顯示 Log	
<code>git log -p FileName</code>		查詢檔案 Log	
<code>git blame FileName</code>		查詢該檔案每行編輯資訊	(上傳者&時間)
<code>git reflog</code>		查詢 reflog	
<code>git help</code>		查詢指令	